

AADK 9: Task 1 — Advanced Conditional Logic & Flow Design

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 9

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Research Key Concepts

if/else Expression vs Statement

An **if/else statement** is used to control program flow by executing different blocks of code depending on a condition. It does not necessarily return a value.

An **if/else expression** evaluates a condition and **returns a value**, which can be assigned directly to a variable. Kotlin treats if as an expression when its result is used.

Example (if/else expression returning value):

```
val amount = 120
```

```
val category = if (amount > 100) {  
    "High Expense"  
} else {  
    "Normal Expense"  
}
```

Here, the if expression returns a string which is stored in category.

when Statement vs Expression

A **when statement** executes different blocks of code based on conditions but does not necessarily return a value.

A **when expression** returns a value and can be assigned directly to a variable. It is often used as a cleaner alternative to multiple if/else statements.

Use of in, is, and Multiple Branch Syntax in when

in

Used to check if a value exists within a range or collection.

is

Used for **type checking**.

Multiple branch syntax

Allows multiple conditions to trigger the same result.

Example: when expression with in and is

```
fun checkValue(value: Any): String {  
    return when (value) {  
        in 0..100 -> "Value is within range"  
        is String -> "Value is a String"  
        else -> "Unknown type or out of range"  
    }  
}
```

This example checks whether the value is **within a range** or a **specific type**.

2. App Feature with Complex Logic

Feature: Expense Tracker – Transaction Categorization

This feature allows users to enter an expense amount and automatically categorizes the transaction. The system validates the amount and determines whether a **discount or surcharge** should be applied depending on the amount and eligibility conditions. The final category and amount are then displayed to the user.

The feature must decide:

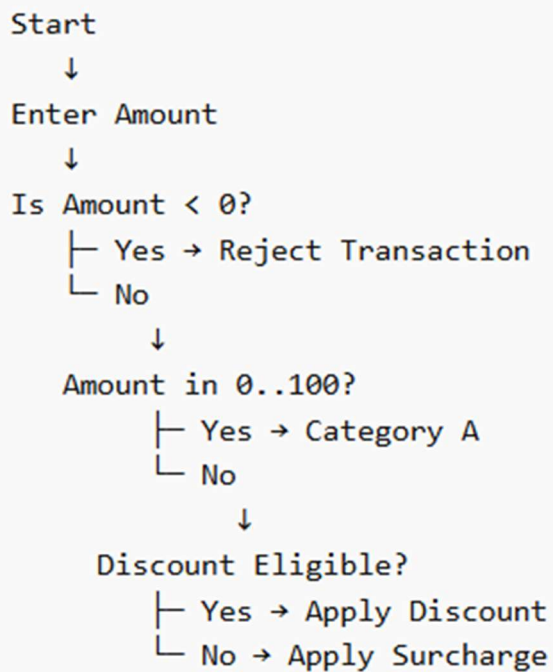
- Whether the amount entered is valid

- Which category the expense belongs to
- Whether a discount or surcharge should apply

3. Decision Flow Diagram / Table

Condition	Action	Code Structure
Amount < 0	Reject transaction (invalid input)	if
Amount in 0..100	Assign Category A (Small Expense)	when with in
Amount > 100 AND discountEligible	Apply discount	if condition
Amount > 100 AND NOT discountEligible	Apply surcharge	else

Flow Representation



Example Kotlin Logic

```
fun processTransaction(amount: Int, discountEligible: Boolean): String {  
  
    if (amount < 0) {  
        return "Invalid transaction"  
    }  
  
    return when (amount) {  
        in 0..100 -> "Category A - Small Expense"  
        else -> {  
            if (discountEligible) {  
                "Discount Applied"  
            } else {  
                "Surcharge Applied"    }    }    }    }  
}
```

4. Compose Integration Plan

In Jetpack Compose, the user will enter the transaction amount using a **TextField**, and a **state variable** will store the input value. When the user presses a **button**, the decision logic function will evaluate the amount and determine the result. The output (category, discount, surcharge, or error message) will be stored in another **state variable**. Compose will automatically update the UI to display the result whenever the state changes.

5. Reflection

Planning complex logic flows before coding is important because it helps developers clearly understand all possible decision paths and edge cases. Without planning, branching logic can become messy and difficult to maintain. Tangled if/else conditions often lead to bugs and confusing code structure. Using expressions like **when** instead of deeply nested if/else improves readability and makes the code easier to maintain and modify in the future.